

Gudumap (SIH26168) — The Complete Technical Guide

Everything the project is, how every piece works, why it was built this way, what broke and how it was fixed, and what's still open. Written so you can defend any part of it to a judge without notes.

Table of Contents

1. The One-Paragraph Pitch
2. The Big Analogy: What Dead Reckoning Actually Is
3. Why This Problem Exists and Why It Matters
4. The Two-Layer Architecture, In Plain Terms
5. The Physics Layer, In Depth
 - 5.1 Coordinate frames (the five "languages" position is spoken in)
 - 5.2 Strapdown inertial navigation (the double-integration trap)
 - 5.3 The Extended Kalman Filter (EKF) — the 6-state estimator
 - 5.4 Zero-Velocity Update (ZUPT) — teaching the system to recognize "I'm standing still"
 - 5.5 Non-Holonomic Constraints (NHC) — teaching the system "cars don't sidestep"
6. The ML Layer, In Depth
 - 6.1 Why ML at all, if physics already works?
 - 6.2 The datasets: OxiOD and IO-VNBD
 - 6.3 Feature engineering — the six numbers the model actually sees
 - 6.4 The GRU model architecture
 - 6.5 Training process, step by step
 - 6.6 The two trained models, and which one is actually used
 - 6.7 Normalization — why raw numbers never touch the network
 - 6.8 The kinematic plausibility gate
7. ONNX: What It Is, Why It's Used, and Exactly Where
8. How GPS Blackout Is Actually Tested (Without Cheating)
9. The Android App (gudumap): Structure and Live Data Flow
10. Evaluation Metrics, Explained Simply
11. The Benchmark Results — Read Honestly
12. Problems Faced and Fixed — The Full Chronological Story
13. Open Challenges and Honest Limitations

14. Jury / Judge Q&A Rehearsal

15. Glossary

1. The One-Paragraph Pitch

Gudumap is an Android app that keeps estimating your position after GPS disappears — in a tunnel, underground parking, a dense urban canyon, or anywhere satellites can't reach the phone. It does this using only sensors already inside every smartphone (accelerometer, gyroscope, magnetometer), combining a physics-based dead-reckoning engine (the reliable, guaranteed backbone) with a small neural network (a GRU, trained on real recorded vehicle and pedestrian motion, exported to ONNX and run entirely on-device) that assists when it's confident and gets ignored when it isn't. Everything runs offline, with no server, no network call, and no cloud dependency — the whole point is that it has to work when connectivity itself may be the thing that's gone.

2. The Big Analogy: What Dead Reckoning Actually Is

Imagine you're blindfolded in a large hall. Someone tells you your exact starting position and which way you're facing. From then on, you're not allowed to peek, but you *can* feel every step you take and every turn your body makes. If you count your steps carefully, know roughly how long each stride is, and track every turn precisely, you can keep a running mental estimate of where you are — without ever seeing the room.

That's dead reckoning. GPS is "peeking" — an outside reference that tells you exactly where you are. The accelerometer and gyroscope are your "feeling your own steps and turns." The problem, and the entire reason this project exists, is that counting steps blindfolded gets less and less accurate the longer you go without peeking — a slightly-too-short stride estimate, repeated for two minutes, adds up to being many meters off from where you actually think you are. This is why **dead reckoning is a bridge, not a replacement** — it's meant to carry you accurately through a short blackout (a tunnel, a covered parking garage) until GPS can peek again, not to navigate a whole city blind.

Sailors have done a version of this for centuries — before satellites, a ship's navigator tracked speed (from a log line dragged behind the ship), heading (from a compass), and elapsed time, and multiplied them out to estimate position on a chart. "Dead reckoning" is even believed to come from that literal practice. This project is the same idea, just done at 10–100 times a second with a phone's inertial sensors instead of a compass and a stopwatch.

3. Why This Problem Exists and Why It Matters

GPS (and India's own NavIC) are line-of-sight satellite systems — they need a relatively clear view of the sky. Tunnels, underground parking structures, dense high-rise "urban canyons," and forested terrain all physically block that line of sight; no amount of software cleverness

fixes a signal that never arrives. This isn't a rare edge case — it happens to essentially every urban commuter, delivery rider, and ride-hailing driver regularly, and it has real consequences: an ambulance's live tracking disappearing exactly when it enters a covered stretch of road; a delivery app's ETA collapsing the moment a rider goes under a flyover; a fleet-tracking system losing continuity through a ghat tunnel.

There's also a timely, India-specific angle: as of mid-2026, NavIC (India's own regional satellite navigation system) has only three operational satellites, while a minimum of four are needed for standalone positioning — the last onboard atomic clock (on IRNSS-1F) failed in March 2026, and the government has told Parliament that NavIC currently cannot provide independent navigation data on its own. NavIC exists specifically to give India navigation capability independent of foreign systems, especially in emergencies — so a satellite-independent fallback layer isn't just a consumer convenience right now, it's filling a live gap in a strategic capability.

Crucially, this problem is **not already solved** for a bare smartphone. Automotive-grade dead reckoning (u-blox ADR, FURUNO GNSS/DR receivers, Sony's GNSS+DR fusion modules) exists and works well, but all of it depends on dedicated automotive-grade IMU hardware and, often, direct access to the vehicle's CAN bus for wheel-tick data — things a phone in your pocket simply doesn't have. Smartphone-only dead reckoning, using nothing but the IMU already inside the phone, is still an active, unsolved research area (published methods from 2021–2025 are still evolving), which is exactly why every major navigation app still visibly glitches or freezes the moment your phone loses satellite lock.

4. The Two-Layer Architecture, In Plain Terms

Think of the system as having a careful, reliable senior engineer (the **physics layer**) and an intuitive, fast-thinking junior engineer (the **ML layer**) working together. The senior engineer always shows up and always gives an answer — a bit conservative, occasionally a little "stiff," but grounded in equations that can't lie. The junior engineer has seen a lot of real driving and walking data and can sometimes spot patterns the senior engineer's equations miss — but the junior engineer is inconsistent: brilliant on some routes, badly wrong on others. So the system's rule is: **the senior engineer's answer is always the fallback**, and the junior engineer only gets to influence the final answer when a separate "sanity checker" (the kinematic plausibility gate) agrees the junior engineer's suggestion is physically reasonable.

Concretely:

- **Physics layer (the reliable backbone):** a strapdown inertial navigation solution feeding a 6-state Extended Kalman Filter (EKF) tracking position and velocity in North-East-Down coordinates, corrected by GNSS whenever it's available, with two physics-based constraints layered on top — Zero-Velocity Update (ZUPT, "you're standing still, stop

drifting") and Non-Holonomic Constraints (NHC, "you're a car, you don't slide sideways or hop vertically").

- **ML layer (the assistive signal):** a GRU (a type of recurrent neural network built for sequences) trained to predict raw 3D displacement directly from a rolling window of raw IMU samples, meant to catch bias/drift patterns physics alone can't model. Exported to ONNX and run entirely on-device. A kinematic plausibility gate sits between the model's raw output and the position estimate, rejecting or clamping any prediction that implies an unrealistic speed change.

5. The Physics Layer, In Depth

5.1 Coordinate frames (the five "languages" position is spoken in)

Position and motion get described in five different coordinate systems as they flow through the pipeline, each one existing because the layer before it needs to hand off to the layer after it in a form that layer understands. This is one of the most conceptually tricky parts of the whole system, so here's the plain-language version first, then the formal one.

Analogy: Think of giving someone directions using only *your own body*: "two steps forward, one step to my right" (that's the **phone/body frame** — relative to how you personally are facing right now). Now imagine converting that into directions relative to the *car* you're sitting in, if you're not facing exactly forward in your seat (the **vehicle frame**). Now convert that into compass directions — "10 meters north, 3 meters east" — so someone else, facing any direction, understands it (the **navigation/NED frame**). Finally, convert that into an actual spot on a world map — a latitude and longitude (the **geodetic/WGS-84 frame**). Each conversion needs to know the *current orientation* of the thing you're converting from, which is exactly what the heading/attitude estimate is for.

The five frames, formally:

1. **Phone sensor frame ($\mathbf{p}$):** the phone's own hardware axes — +X points right across the screen, +Y points up along the screen (toward the top speaker), +Z points out of the screen toward your face. This is the frame every raw accelerometer/gyroscope reading arrives in.
2. **Vehicle body frame ($\mathbf{b}/\mathbf{v}$):** standard automotive convention — +X forward (direction of travel), +Y right (lateral), +Z down (toward the road). A fixed rotation matrix, $\mathbf{R}_{pv}$ (phone-to-vehicle), converts phone-frame readings into this frame; by default this is just the identity matrix (assumes the phone is mounted flush in landscape on the dashboard with its axes already aligned to the car's).
3. **Local window frame ($\mathbf{b}_o$):** the vehicle's own orientation *at the start of each inference window* (frozen for that window's duration). This is the frame the ML model's $\begin{bmatrix} dx, & dy, \end{bmatrix}$

$\overline{dz}]$ output is expressed in — it doesn't know or care about compass directions, only "relative to which way I was facing when this 2-second window began."

4. **Navigation frame / NED (n):** a local flat-Earth tangent plane anchored at a reference point (your starting GNSS fix), with +North, +East, +Down axes. This is the frame the EKF's internal state actually lives in.
5. **Geodetic frame / WGS-84 (g):** true latitude, longitude, and altitude on the real globe — what finally gets drawn on the map.

Every step between frames 3 and 5 needs the current **heading** (compass bearing, clockwise from true North, 0° – 360°), because rotating a "10 meters forward" vector into "10 meters north-east" literally requires knowing which way "forward" was pointing. This is why heading quality is so central to the whole system — an error in heading doesn't just misplace you, it misdirects every future displacement estimate too, since the app keeps adding new "forward" vectors on top of a wrongly-rotated base.

5.2 Strapdown inertial navigation (the double-integration trap)

"Strapdown" just means the sensors are physically bolted to (strapped down inside) the device, rather than mounted on a mechanically-stabilized gimbal like old-school ship/aircraft INS — modern phones and cheap MEMS chips are strapdown by necessity, and the "un-stabilizing" has to happen entirely in software.

The classical physics chain is:

1. **Orientation tracking:** integrate the gyroscope's angular-rate readings over time to know which way the device is currently facing (fused with the accelerometer/magnetometer for drift resistance — see 5.1's heading discussion).
2. **Specific-force rotation:** rotate the raw accelerometer reading from the device frame into the navigation frame using that orientation, then subtract gravity to isolate the *true* linear acceleration (the part actually caused by you moving, not by gravity pulling on the sensor).
3. **Double integration:** integrate acceleration once to get velocity, integrate velocity again to get position.

Step 3 is the trap, and it's worth understanding exactly why, because it's the reason almost everything else in this document exists. Accelerometers are noisy — every real reading has a small random error on top of the true value. Integrating once (to velocity) turns that noise into a **random walk** in the velocity estimate — it doesn't cancel out over time, it *accumulates*. Integrating a second time (to position) turns that random walk into something that grows even faster: position error compounds **quadratically** with time for a *constant* bias, and even for pure zero-mean noise, still grows unboundedly rather than averaging to zero, because each integration step is added on top of *all* the error accumulated so far. In numbers: a tiny,

seemingly negligible acceleration bias, uncorrected, produces a position error that grows with the *square* of elapsed time — twice as long without a correction isn't twice the error, it's four times the error. This is exactly what the "naive trail" in the app's blackout view is showing you on purpose (see §9 and §12) — it's not a demo gimmick, it's the textbook failure mode of raw dead reckoning made visible.

Every other physics component in this project — the EKF, ZUPT, NHC — exists specifically to fight this one compounding-error problem, by injecting outside information (a Kalman correction, a "you're stationary" pseudo-measurement, a "you can't be sliding sideways" pseudo-measurement) often enough that the quadratic blowup never gets the chance to run away.

5.3 The Extended Kalman Filter (EKF) — the 6-state estimator

This project's own documentation is explicit and honest about what this filter is: a **simplified 6-state MVP navigation filter**, not a full 15-state error-state INS (which would also estimate accelerometer bias, gyroscope bias, and attitude error as part of the state itself). That's a real, disclosed scope limitation, not an oversight — described in more detail in §13.

State vector (what the filter is tracking, 6 numbers total):

$$\mathbf{x} = [p_N, p_E, p_D, v_N, v_E, v_D]^T$$

— position North/East/Down (meters, relative to an anchor point) and velocity North/East/Down (m/s). Alongside the state is a 6×6 **covariance matrix** $\mathbf{P}$, which is the filter's own "how confident am I in each of these numbers, and how are my errors in one number related to my errors in another" — this is what eventually becomes the widening "confidence radius" you see on screen during a blackout.

Analogy for the whole EKF loop: imagine you're estimating a friend's weight by guessing, then repeatedly stepping on a slightly-unreliable bathroom scale. Each time, you don't just throw away your previous guess and trust the scale completely, and you don't ignore the scale either — you blend the two, weighted by how much you currently trust each one. If your previous guess was very uncertain and the scale is usually reliable, you lean heavily toward the new scale reading. If your guess was already very confident and the scale seems to be acting up, you barely move your estimate. That blending ratio is the **Kalman gain**, and every "measurement update" below is one instance of this blend.

The filter runs in a **predict → update** loop:

- **Predict (time update):** every time a new displacement increment arrives (from the ML model, or from raw IMU integration), the state moves forward — $\mathbf{p}_k = \mathbf{p}_{\{k-1\}} + \Delta \mathbf{p}$, and velocity is derived as $\Delta \mathbf{p} / \Delta t$. Because this step is itself an estimate (not a perfect

measurement), uncertainty is *added* to the covariance every time this runs — this is the mathematical source of "confidence radius grows the longer you go without a correction."

- **Update (measurement update):** whenever a trustworthy outside measurement is available (GNSS position, GNSS velocity, a ZUPT "I'm stationary" pseudo-measurement, an NHC "I'm not sliding sideways" pseudo-measurement), the filter computes the **innovation** (how far off the current estimate is from what this measurement says), weighs it by the Kalman gain, and nudges the state toward the measurement — while *shrinking* the covariance, since a correction just happened.

The exact numbers, so you can quote them precisely if asked: initial position uncertainty $\sigma_{\text{pos}} = 3.0$ m, initial velocity uncertainty $\sigma_{\text{vel}} = 0.3$ m/s; process noise (how much uncertainty is injected per prediction step) $q_{\text{pos}} = 0.05$ m, $q_{\text{vel}} = 0.20$ m/s; GNSS position measurement noise $\sigma \approx 2.5\text{--}3.0$ m; GNSS velocity measurement noise $\sigma \approx 0.2\text{--}0.3$ m/s. The covariance update uses the numerically-stable **Joseph form** ($P = (I - KH)P(I - KH)^T + KRK^T$) rather than the simpler textbook form, specifically because the simpler form can drift into a mathematically invalid (non-positive-definite) covariance matrix after thousands of update cycles due to floating-point rounding — Joseph form is guaranteed to stay valid regardless.

During a GNSS blackout, the GNSS position/velocity updates are simply skipped — the filter keeps predicting forward using ML/IMU displacement increments, constrained by ZUPT and NHC, with covariance (and therefore the on-screen confidence radius) growing the whole time. When GNSS returns, updates resume and the Kalman gain smoothly blends the estimate back toward the fresh fix — deliberately avoiding a jarring, discontinuous "snap" on the map.

5.4 Zero-Velocity Update (ZUPT) — teaching the system to recognize "I'm standing still"

Analogy: if you're counting your own footsteps blindfolded and you know for certain you just stood still at a red light for 20 seconds, the smart thing to do is reset your "current speed" belief to exactly zero for those 20 seconds, rather than let small phantom "steps" (your hand shaking slightly, the car idling and vibrating) keep silently adding to your distance-traveled count. ZUPT is exactly that reset, applied automatically whenever the sensors say you're genuinely still.

A **multi-signal detector** looks at a short rolling window of recent IMU samples and only declares "stationary" when *all* of the following hold simultaneously:

- Acceleration magnitude below 0.25 m/s^2 ($\approx 0.08 \text{ g}$) **and** its variance below $0.04 (\text{m/s}^2)^2$ — i.e., not just momentarily small, but *consistently* small.
- Gyroscope magnitude below 0.10 rad/s **and** its variance below $0.01 (\text{rad/s})^2$ — you're not rotating either.
- GNSS speed below 0.30 m/s , when a GPS fix is available (an extra cross-check).

- A **debounce**: this condition has to hold for a minimum duration (currently time-based, not sample-count-based — see the ZUPT bug in §12) before the system actually commits to "stationary," to avoid a single lucky quiet instant mid-stride from falsely triggering it.

Once confirmed stationary, ZUPT feeds the EKF a **pseudo-measurement**: "velocity = [0, 0, 0]," with its own small measurement noise ($\sigma \approx 0.02\text{--}0.05$ m/s). Critically — and this is stated explicitly and repeatedly in the project's own frozen documentation — velocity is **never hard-clamped or directly overwritten**. It only ever gets corrected the same way any other measurement corrects the filter: through the normal Kalman gain math. This matters because a hard clamp would be a lie about how confident the system actually is; going through the Kalman update keeps the covariance mathematically honest.

5.5 Non-Holonomic Constraints (NHC) — teaching the system "cars don't sidestep"

Analogy: a car (unlike, say, a shopping cart with all four wheels able to swivel) can only really move forward/backward along the direction it's pointed — it doesn't glide sideways, and normally it doesn't bounce up and down either. If your position estimate ever implies the car briefly drifted sideways relative to its own heading, that's almost certainly sensor noise being misread as real motion, not an actual sideways slide.

NHC exploits exactly this physical fact as a second pseudo-measurement: it asserts that the vehicle's **lateral** velocity (sideways, relative to its own heading) and **vertical** velocity (up/down) should both be zero during normal driving:

```
z_nhc = [v_lateral, v_vertical] = [0.0, 0.0]
```

This gets projected from the filter's North-East-Down velocity state using the current heading ψ via the observation matrix $\mathbf{H}_{\text{nhc}}$, with its own measurement noise ($\sigma_{\text{lateral}} = 0.15$ m/s, $\sigma_{\text{vertical}} = 0.10$ m/s). Like ZUPT, this is a soft Kalman correction, not a hard override, and it's explicitly scoped to "normal forward wheeled driving" — it assumes no significant tire slip, drifting, or big vertical bounce, and the project's documentation is upfront that dynamic tire-sideslip modeling (e.g., for genuinely aggressive driving) is **not implemented**.

6. The ML Layer, In Depth

6.1 Why ML at all, if physics already works?

Physics-only dead reckoning is honest and grounded, but it has a specific weakness: raw MEMS accelerometers and gyroscopes have small systematic biases and non-linear noise characteristics that are hard to model with clean equations — the kind of pattern a model trained on thousands of real recorded seconds of "what this exact sensor class actually outputs when a real car goes over this exact kind of bump" can sometimes pick up on, where a fixed physics formula can't. The idea (matching the two-layer analogy from §4) is that the ML model

might catch bias/drift signatures that are genuinely learnable from data but aren't captured by the strapdown+EKF math alone — *if* it can be trusted, and *only* when it can be trusted, which is exactly what §6.8's gate exists to arbitrate.

6.2 The datasets: OxIOD and IO-VNBD

Two real, published, peer-reviewed datasets sit behind the two trained models — nothing about the underlying sensor data is synthetic.

OxIOD (Oxford Inertial Odometry Dataset) — the pedestrian/handheld dataset. 158 sequences, 42.5 km / 14.7 hours of smartphone IMU data across 5 users and 4 phone-carrying placements (handheld, pocket, handbag, trolley), with **millimeter-accurate ground truth** from a Vicon motion-capture rig — a room instrumented with multiple precision cameras that track reflective markers on the person in real time, giving a "ground truth" position accurate enough to trust as the training target. Recorded on an iPhone 7/SE held in standard portrait orientation.

IO-VNBD (Inertial and Odometry Vehicle Navigation Benchmark Dataset) — the vehicle dataset, and the one actually deployed. This is the dataset ISRO linked directly from the official SIH portal for this problem statement. Audited directly by the project (not taken on faith): 144 smartphone CSV files + 144 matching vehicle-ECU CSV files, synchronized into **72 matched recording pairs**, covering **29.74 hours** of real driving (1,070,640 total samples) across **5 different drivers**, sampled natively at **10 Hz**, with mean vehicle speed 45.08 km/h and a max of 131.85 km/h — this is genuine CAN-bus ground truth (the vehicle's own onboard electronics reporting real speed and, in the offline benchmark, heading), not a simulated trajectory.

How the phone was actually mounted matters, and was verified, not assumed: auditing the raw CSVs showed the phone was mounted in landscape orientation on the dashboard, with gravity concentrated on the phone's Z-axis. A correlation analysis between the phone's own gyroscope channels and the vehicle's CAN-bus-reported yaw rate found the phone's **pitch-axis** gyroscope correlates with real vehicle turning yaw rate at $r = 0.9348$ — a very strong correlation — which is what justified mapping phone-pitch-gyro into the model's "yaw rate" feature slot for this dataset.

Leakage-free splitting — a detail that matters a lot for trustworthiness: splits were made by **entire physical driving sequence**, not by randomly shuffling individual windows, specifically so that windows from the same drive never end up split across train and test. IO-VNBD: 53 sequences / 45,742 windows for training, 10 sequences / 33,262 windows for validation, 9 sequences / 27,964 windows held out for testing — with each split drawing from different drivers/routes where possible, to genuinely test generalization rather than memorization of a specific route.

6.3 Feature engineering — the six numbers the model actually sees

Both models take exactly the same **six input channels**, in this exact order, every time:

#	Feature	What it physically means	Unit fed to the model
0	<code>acc_x</code>	Linear (gravity-removed) acceleration, device/vehicle X-axis	g (1 g = 9.80665 m/s ²)
1	<code>acc_y</code>	Linear acceleration, Y-axis	g
2	<code>acc_z</code>	Linear acceleration, Z-axis	g
3	<code>gyro_x</code>	Angular velocity, X-axis (vehicle turning/yaw rate, for IO-VNBD)	rad/s
4	<code>gyro_y</code>	Angular velocity, Y-axis	rad/s
5	<code>gyro_z</code>	Angular velocity, Z-axis	rad/s

Two details here are the source of the single most dangerous, easy-to-make mistake in the whole deployment, so they're worth internalizing precisely:

Gravity must already be removed before the model ever sees the acceleration. Android's `Sensor.TYPE_ACCELEROMETER` (raw) includes Earth's gravity (~9.8 m/s² baked into whichever axis is "up"); `Sensor.TYPE_LINEAR_ACCELERATION` has gravity already subtracted out by Android's own sensor fusion. The model was trained exclusively on gravity-removed data (OxIOD's own `user_acc_*` columns, explicitly distinct from its separate `gravity_*` columns) — feeding it raw, gravity-included acceleration would inject a huge, constant, nonsensical "acceleration" into every single window.

Units must be converted, not just correctly sourced. Android's linear-acceleration sensor reports in m/s². The model was trained on data in g. Forgetting the division by 9.80665 scales every acceleration input by roughly 9.8×, which doesn't crash anything — it just silently produces a badly wrong displacement estimate that *looks* like a plausible number. This exact risk is called out, in capital letters, in the project's own deployment contract documentation — it's the single most-flagged failure mode in this entire codebase's docs, precisely because it's the kind of bug that doesn't throw an error, it just quietly lies.

Gyroscope units need **no conversion** — Android's `TYPE_GYROSCOPE` and the model's training data are both already in rad/s.

6.4 The GRU model architecture

What a GRU is, in plain terms: a normal ("feedforward") neural network looks at one snapshot of input and produces one output, with no memory of what came before. A **recurrent** network,

by contrast, is built for *sequences* — it reads the input one time-step at a time, and carries forward a small internal "memory" (a hidden state) that gets updated at every step, so its output at any point can depend on everything it's seen in the sequence so far, not just the current instant. A **GRU (Gated Recurrent Unit)** is a specific, well-established design for that hidden-state update that uses small internal "gates" to decide, at each step, how much of the old memory to keep versus how much of the new input to blend in — this is what lets it learn, for example, "a sharp acceleration spike followed by three seconds of steady vibration usually means highway driving, not a pothole," a pattern that only makes sense across several time-steps, not from any single one.

The analogy: imagine reading a sentence one word at a time and trying to guess where the sentence is going. A model with no memory can only look at the current word in isolation. A recurrent model is like a reader who genuinely remembers everything read so far in the sentence and updates their understanding word by word — which is exactly why GRUs (and their cousin, LSTMs) became the standard choice for any raw sensor time-series problem before newer sequence architectures existed, and remain a very solid, efficient, easy-to-deploy choice for a short, fixed-length window like this one.

This project's exact architecture (`GRUDeadReckoning`), from (`src/models/gru_model.py`):

- **Input size:** 6 (the six IMU channels above)
- **Hidden size:** 64 (each GRU layer's internal memory is a vector of 64 numbers)
- **Number of layers:** 2, **stacked** (the first GRU layer's output sequence feeds directly into a second GRU layer, letting the network build a more abstract representation before producing an answer — much like stacking two ordinary neural-network layers)
- **Dropout:** 0.2 (during training, 20% of internal connections are randomly zeroed out on each pass — a standard regularization trick that stops the network from over-relying on any single internal pathway, which reduces overfitting)
- **Output size:** 3 — a single `[dx, dy, dz]` displacement vector for the *entire window*, not a prediction at every time-step. The network reads the whole 2-second window, then produces one final answer: "here's the net displacement over that window."

6.5 Training process, step by step

1. **Raw data in, features out:** each dataset's raw CSVs are loaded, gravity-removed acceleration and gyro rates are extracted and reordered into the exact 6-column contract above, and any dataset-specific axis remapping (like the pitch→yaw mapping found via the $r = 0.9348$ correlation for IO-VNBD) is applied.
2. **Windowing:** the continuous stream is cut into fixed-length overlapping windows — for OxIOD (`gru_local`), 200 samples at 100 Hz (a 2.0-second window), stride 200 (non-overlapping) for training; for IO-VNBD (`gru_io_vnbd`), 20 samples at its *native* 10 Hz

(also a 2.0-second window, since $20 \times 0.1 \text{ s} = 2.0 \text{ s}$), with a 10-sample (50% overlap) stride, giving more frequent updates during inference.

3. **Ground-truth target per window:** the true displacement the object actually underwent during that window, computed from whatever ground-truth source that dataset provides (Vicon motion capture for OxIOD; CAN-bus/GPS-derived ground truth for IO-VNBD), expressed in the **local frame at the start of the window** — $\overline{y = R_{\text{start}}^T \cdot (p_{\text{end}} - p_{\text{start}})}$. This is why the model itself never needs orientation as an input: the target itself is already defined relative to "wherever you were facing when this window began," so the network only ever has to learn "how far did this pattern of accelerations and rotations carry me, relative to my own starting facing direction" — a self-contained regression problem that doesn't require knowing compass north.
4. **Normalization statistics computed strictly from the training split** (see §6.7) — never from validation or test data, to avoid leaking any information about the held-out sequences into how the input/output are scaled.
5. **Optimizer and loss:** AdamW (a standard, well-regularized variant of the Adam optimizer) with a learning rate of $1\text{e-}3$ and weight decay of $1\text{e-}4$, a cosine-annealing learning-rate schedule (the learning rate smoothly decreases over training rather than dropping in sudden steps), and **Smooth L1 loss** (a loss function that behaves like squared error for small mistakes but like absolute error for large ones — this makes training more robust to occasional large-outlier windows, like a pothole hit, without letting them dominate the whole training signal the way plain squared error would).
6. **Transfer learning between the two models:** the IO-VNBD (vehicle) model was **not trained from a random starting point** — it was initialized from the already-trained OxIOD (pedestrian) model's weights, then fine-tuned on vehicle data. This is a standard and sensible move: the low-level "how do IMU signals over a couple of seconds relate to real displacement" patterns learned from pedestrian data are a reasonable starting point even for a very different motion domain, and starting from them (rather than from scratch) tends to need less data and fewer epochs to converge well — genuinely useful given the vehicle dataset, while large in hours, still represents far fewer *distinct* driving conditions than would ideally be wanted.
7. **Early stopping:** training runs for a capped number of epochs, but the checkpoint actually kept is whichever epoch had the best *validation*-set loss, not the very last epoch — this guards against a network that keeps improving on training data while quietly getting worse at generalizing. For the vehicle model, the best checkpoint was actually epoch 1 out of a run (train loss 0.1859, val loss 0.2217) — an honestly early stopping point, consistent with fine-tuning from an already-reasonable starting point rather than training from scratch.
8. **Random seed fixed (42)** for reproducibility — so a rerun of the exact same training recipe on the exact same data produces the same result, which matters for being able to

defend "this number is real and repeatable," not a one-off lucky run.

6.6 The two trained models, and which one is actually used

This is a detail worth being completely precise about, because it's the kind of thing a technically sharp judge will specifically probe for, and the honest answer is more interesting than a vague one:

	<code>gru_local.onnx</code>	<code>gru_io_vnbd.onnx</code>
Trained on	OxIOD (pedestrian, handheld)	IO-VNBD (real vehicle, CAN-bus ground truth)
Input shape	<code>[1, 200, 6]</code>	<code>[1, 20, 6]</code>
Native sampling rate	100 Hz	10 Hz
Window duration	2.0 s (200 samples)	2.0 s (20 samples)
Update stride	200 (non-overlap) in training; app can use overlap	10 samples (1.0 s, 50% overlap)
Typical output scale	~1.29 m average displacement per window (walking)	~26.47 m average displacement per window (driving) — a 20× scale difference , which is exactly why each model needs its own separate normalization statistics, not a shared one
Actually wired into the shipped Android app?	No. Present in <code>assets/</code> but never loaded by any Kotlin code.	Yes. <code>ModelRunner.kt</code> hardcodes this file as its load target.

So: two real, separately-trained, separately-validated models exist, but only the vehicle model is currently live in the product. The pedestrian model isn't fake or abandoned — it's a genuinely trained, genuinely evaluated artifact sitting ready for a future integration phase — but it would be inaccurate to describe the shipped app as running "two fused ONNX models." (How this was actually discovered — via a careful repo-wide grep rather than assumption — is worth knowing for its own sake; see §12.)

6.7 Normalization — why raw numbers never touch the network

Neural networks train and predict far better when every input feature is on a comparable numeric scale — without this, a feature that happens to have naturally larger raw values (say,

gyro readings during a sharp turn) can dominate the learning signal purely because of its scale, not because it's actually more important. So every raw 6-value sample is **standardized** before it ever reaches the network:

```
x_normalized = (x_raw - mean) / std
```

using **mean and standard deviation computed only from that model's own training split** (never validation/test — see §6.5, point 4). The network's raw output then has to be **denormalized** back into real meters the same way, in reverse:

```
displacement_meters = y_raw * target_std + target_mean
```

The exact frozen numbers for the deployed vehicle model (`io_vnbd_normalization.json`), for reference:

- Input mean: `[0.00548, 0.00832, 0.01211, 0.00312, 0.00194, 0.00287]`
- Input std: `[0.08124, 0.07651, 0.09342, 0.04512, 0.03891, 0.04120]`
- Target mean (meters): `[26.4712, 0.0213, 0.0000]`
- Target std (meters): `[16.9550, 1.4200, 1.0000]`

Notice the target mean's first value, ~26.47 m — that's the average forward displacement per 2-second window across the training drives, which only makes sense for vehicle speeds (roughly 47 km/h average), underlining why this model's normalization is not interchangeable with the pedestrian model's.

6.8 The kinematic plausibility gate

Analogy: imagine a junior engineer (the ML model) hands you a displacement estimate every second, and before you accept it, a supervisor (the gate) does one quick sanity check: "does this imply the car just changed speed by a physically ridiculous amount in one second?" If yes, the supervisor throws that estimate out and falls back to the trusted physics-only estimate for that one moment, rather than letting a single bad guess corrupt the whole trajectory.

Concretely, the gate looks at what displacement the model just predicted, works out what instantaneous speed/speed-change that implies, and classifies the prediction into one of three outcomes:

- **ACCEPTED** — the implied motion is kinematically reasonable; the ML displacement is used.
- **CLAMPED** — the direction is plausible but the magnitude is too aggressive; the displacement is scaled down toward something physically sane rather than thrown away entirely.

- **REJECTED** — the prediction is kinematically implausible (an impossible speed jump); it's discarded for this step, and the system falls back to physics/IMU-only propagation for that step instead.

This gate is the mechanism that lets the ML model contribute *only* when it's earned trust for that specific moment, which is exactly the safety net that made it possible to ship an ML layer honestly even after discovering (see §11 and §12) that the ML model doesn't reliably beat plain physics on every route.

7. ONNX: What It Is, Why It's Used, and Exactly Where

Analogy: think of ONNX (Open Neural Network Exchange) as a universal power adapter. The model is *trained* in PyTorch, on a full desktop/laptop with a real Python environment, GPU support, and every convenience a researcher needs. But the place the model needs to actually *run* — a phone, offline, with no Python installed, needing to answer in under a millisecond, dozens of times a second, without draining the battery — is a completely different environment. ONNX is the standard, portable file format that lets a model trained in one framework (PyTorch here) be "plugged in" and executed efficiently by a completely different, lightweight runtime (ONNX Runtime) on a completely different platform (Android/Kotlin), without needing PyTorch, Python, or any of the training-time machinery to be present at all.

Why not just ship PyTorch itself onto the phone? PyTorch is a large, general-purpose research framework — running it directly on a phone would be heavyweight, slow to start, and unnecessary, since inference (just running the already-trained network forward) needs almost none of PyTorch's training machinery. ONNX Runtime, by contrast, is a small, highly optimized engine built specifically for *running* (not training) models fast, including on mobile CPUs.

Exactly where it's used in this project:

- **Export step:** after training in PyTorch (`.pt` checkpoint files), the model is converted to the `.onnx` format via `src/models/export_onnx.py` (OxIOD model) and `src/models/export_io_vnbd_onnx.py` (vehicle model), at **opset 17** (opset = the specific version of ONNX's own operator set the file targets, similar to specifying which language standard a piece of code was compiled against).
- **Parity verification step** (`src/models/verify_onnx.py`): the exported ONNX model is run on the exact same test inputs as the original PyTorch checkpoint, and the two outputs are compared. This isn't a formality — it's the check that catches "the conversion silently changed the model's behavior." The measured result: a maximum absolute difference of **1.192×10^{-6} meters** between PyTorch and ONNX outputs (mean difference $\sim 9.2 \times 10^{-8}$ m) — effectively floating-point noise, confirming the exported model behaves identically to the one that was actually trained and validated.

- **On-device inference** (`ModelRunner.kt`): the Android app loads `gru_io_vnbd.onnx` via **ONNX Runtime Mobile**, feeds it the normalized `[1, 20, 6]` input tensor named `imu_window`, and reads back the `[1, 3]` output tensor named `local_displacement` — entirely locally, entirely offline, with zero network calls at any point in the pipeline.
- **Measured performance**: mean inference latency of **0.177 ms**, median 0.159 ms, P95 0.232 ms on CPU — over 5,600 inferences per second of raw capacity, against a requirement of roughly one inference per second. That's more than 1,000× the headroom actually needed, which is why running this model continuously has a negligible battery/CPU cost.
- **Immutability / freeze discipline**: the production ONNX file, its matching PyTorch checkpoint, its normalization JSON, and its metadata JSON are each **SHA-256 hashed and archived** in a `models/frozen_io_vnbd/` folder specifically so that a build pipeline can verify the exact bytes being shipped in the APK match the exact model that was actually benchmarked — preventing an accidental silent retrain or "quick tweak" from invalidating every number in this document without anyone noticing.

8. How GPS Blackout Is Actually Tested (Without Cheating)

You can't literally drive into a real tunnel every time you want to test a new version of the model — so blackout performance is tested using a standard, accepted technique from the dead-reckoning research literature, and it's worth being able to explain precisely why this isn't "faking" the evaluation.

Analogy: imagine testing how well someone can navigate blindfolded by first having them walk a route *with* a GPS tracker recording their real position the whole time (no blindfold yet). Afterward, you take that real recording and simply **hide** — don't fabricate, hide — a chunk of it, say from the 40-second mark to the 100-second mark. Then you replay the sensor data through the blindfolded-navigation algorithm for that same stretch and see how far its guess ends up from the *real, already-recorded* position you deliberately hid. Nothing about the person's actual walk was invented — you just chose which already-true seconds to test against blind.

That's exactly the process: take a real drive/walk where GPS was available the whole time, **withhold (mask) a window of the real GPS labels** to simulate a blackout, run the dead-reckoning pipeline through that window with GPS hidden, then compare the pipeline's estimate against the real, previously-recorded ground truth that was withheld — never fabricated. The accelerometer and gyroscope readings feeding the pipeline during the "blackout" are always the genuinely recorded real sensor data; the *only* simulated element is the decision of which time window to treat as blacked-out. This project states that distinction explicitly and proactively (rather than waiting to be asked), and one genuine bug was found and fixed in this exact area — a **ground-truth heading leak**, where an earlier version of the evaluation code was accidentally still reading live ground-truth heading *through* the simulated

blackout instead of freezing it at the moment the blackout began, which had been artificially inflating accuracy numbers (see §12).

9. The Android App (gudumap): Structure and Live Data Flow

Tech stack: Kotlin + Jetpack Compose (min SDK 24 / target SDK 37), with an offline-first map built on the **MapLibre Android SDK** rendering **vector tiles** (MVT — Mapbox Vector Tile format) from a bundled offline tile package for a demo region (currently Coimbatore, zoom levels 11–16), styled via a custom `style_template.json`. Everything — ML inference, EKF fusion, ZUPT, map rendering — runs entirely on-device; there is deliberately no backend server in the shipped product, since the entire point of the app is to keep working when connectivity (which a server would need) may itself be the thing that's gone.

The live wiring, traced module by module:

```
MainActivity
→ ui/screens/NavigationScreen.kt
→ viewmodel/NavigationViewModel.kt
→ navigation/NavigationEngine.kt
→ navigation/DeadReckoningEngine.kt
    (EKF, ZuptDetector, NHC, IMUBuffer, CoordinateTransformer,
    ml/ModelRunner, ml/InputNormalizer, ml/ModelMetadata,
    sensor/DiagnosticRecorder, tracking/TrajectoryIntegrator)
→ sensors/{SensorManager, SensorFusionManager, LocationManager}
→ map/{MapMatcher, OfflineMapManager}
```

What the live screen actually shows: GNSS/EKF/ML/Map status cards, a position-confidence card (the EKF covariance made visible), a vehicle-vs-pedestrian motion-mode badge (drawn from a rolling GNSS speed history — fast and steady enough recently classifies as `VEHICLE_MODE`, otherwise `CONSERVATIVE_MODE`/pedestrian), sensor-active indicators, a GNSS-blackout toggle for demoing the whole system without physically finding a tunnel, and the map itself with a live, heading-rotated vehicle marker.

The dual-trail blackout visualization is one of the most genuinely useful things this app does for making the abstract concept of "dead reckoning working" visible in real time: during a blackout, it draws **two separate trails simultaneously** — a **naive (red) trail**, pure double-integration of raw accelerometer data with *no* ZUPT, EKF, or ML correction at all, deliberately left uncorrected as a live reference of what "doing nothing smart" looks like (see §5.2's explanation of why this drifts so badly), and a **corrected (blue) trail**, the actual fused EKF+ML+ZUPT+NHC estimate. Watching the two visibly diverge, live, is the single clearest way to demonstrate — to a judge or to yourself — that the correction pipeline is doing real, meaningful work, rather than asking anyone to trust a number in a table.

10. Evaluation Metrics, Explained Simply

Metric	Plain-language meaning	Analogy
ATE (Absolute Trajectory Error)	The overall root-mean-square distance between your estimated path and the real path, over the whole test segment.	"On average, across the whole trip, how far off the true route was your guessed route?"
RTE (Relative Trajectory Error)	Same idea, but measured over short fixed-length chunks (e.g., every 60 seconds) rather than the whole trip — isolates <i>short-term</i> drift behavior from <i>long-run</i> accumulation.	"In any given minute, how much extra error did you pick up, regardless of how lost you already were before that minute started?"
CEP (Circular Error Probable), e.g. CEP-50 / CEP-90	The radius of a circle, centered on the true position, within which your estimate falls X% of the time.	"Draw a circle around the true spot — CEP-90 is how big that circle needs to be so that 9 times out of 10, your guess landed inside it."
Drift rate (%)	Final position error divided by total distance actually traveled, as a percentage.	"For every 100 meters you actually drove, how many meters off course did you end up?" — the standard way this number is reported across the automotive/robotics dead-reckoning literature, which is exactly why this project reports it the same way, to make direct comparison against published numbers possible.

11. The Benchmark Results — Read Honestly

The project built a full **7-baseline navigation ladder** (physics-only through fully-combined) and ran it across **9 completely held-out IO-VNBD test drives** at **10s, 30s, 60s, and 120s** simulated blackout durations — 224 total evaluation runs, avoiding the trap of reporting one cherry-picked "best" drive.

The ladder:

- **B1 — Pure INS:** raw strapdown double-integration only, no corrections at all.
- **B2 — INS + EKF:** the physics baseline described in §5, with GNSS fusion when available.

- **B3 — ML Only:** the GRU model's displacement predictions alone.
- **B4 — ML + INS:** ML displacement fed through the basic INS propagation.
- **B5 — ML + INS + EKF:** ML displacement fed through the full EKF.
- **B6 — ML + INS + EKF + NHC:** adds the non-holonomic constraint.
- **B7 — ML + INS + EKF + NHC + ZUPT:** the full stack, everything combined.

The uncomfortable, important finding, stated plainly rather than buried: across the full 224-evaluation benchmark, **plain physics (B2, INS+EKF) frequently matches or beats every ML-inclusive configuration**, at multiple outage durations, on the *full* dataset including stationary/idling intervals — for example at 10s duration, B2's median drift was 12.99% versus B3 (ML Only)'s 35.42%; at 30s, B2's median was 35.03% versus B3's 50.82%. Restricting to genuinely *moving* sequences only (ground truth displacement > 100 m — a fairer comparison, since a parked car with engine vibration is a scenario ML specifically struggles with, discussed below) narrows the gap and even flips it at longer durations: at 120s on moving-only sequences, B4 (ML+INS) achieved a **28.62% mean / 22.22% median** drift versus B2's 64.21% mean / 81.16% median — a real, meaningful win for the ML-assisted pipeline on sustained highway-style driving specifically.

So the honest summary is nuanced, not a clean "ML wins" or "ML loses": the ML model is not broken (a specific investigation ruled out a code bug — see §12) — it's **genuinely inconsistent across driving conditions**, doing very well on some real sequences (the best moving drive, vw16a, hit just 9.10% drift at a full 120 seconds of blackout) and badly on others (the worst moving drive, vw14c, hit 72.77% at the same duration), with a median moving drift of 25.79% across all held-out sequences. One specific, well-understood failure mode was identified directly: on vw15 (a sequence where the vehicle was parked with the engine idling, true displacement only 0.04 m), the ML model predicted a spurious ~3.28 m of "displacement" purely from chassis vibration — exactly the kind of scenario ZUPT exists to suppress, and exactly why B7 (with ZUPT) recovers substantially over B6 (without it) specifically on drives containing stops: on one 60-second stop-containing interval, adding ZUPT cut drift from 651.80% down to 264.16%.

Given this finding and the competition's time constraints, the deliberate, disclosed decision was: make physics-based EKF fusion the **guaranteed default** position estimate, with the ML model contributing only when its own kinematic plausibility gate (§6.8) actively trusts a specific prediction — rather than either quietly hiding the ML work, or shipping something measurably worse than the simpler alternative and hoping nobody checks. This is a genuinely defensible engineering call, and being able to explain *why* it was made (not just that it was) is worth having ready for a judge (see §14, Q4 and Q9).

12. Problems Faced and Fixed — The Full Chronological Story

This project went through more than one real technical crisis, and being able to narrate them honestly — what broke, how it was found, how it was actually fixed — is worth more in front of a judge than any polished-after-the-fact summary. In order:

1. The first prototype had zero real data behind it, and was quietly abandoned. An early Python/FastAPI prototype (`(SIH26168-DeadReckoning/)`, physics-baseline + LightGBM-residual design) was independently audited and found to have collected no real sensor traces at all, never trained its LightGBM model on any real data, ZUPT thresholds that hadn't been validated in either direction that was tried, and a backend that was **fully mocked** — every API response was self-labeled `("source": "mock")`. Rather than patch around this, it was formally deprecated; only its evaluation math (`(metrics.py)`, later independently verified as correctly implementing ATE/CEP/RTE/drift-rate) was kept and carried forward into the real pipeline, precisely because trustworthy metric formulas are still useful even when the data and model around them weren't.

2. Early exploratory work interpolated 10 Hz vehicle data up to 100 Hz — this was deliberately rejected. To reuse a single shared 200-sample-window model contract, IO-VNBD's native 10 Hz recordings were initially stretched to 100 Hz via linear spline interpolation. This was caught and reversed before production: interpolation *fabricates* nine artificial samples between each real measurement, which smooths away exactly the high-frequency road vibration and shock transients that carry real information, and running 100 inferences/second when the physical sensor only actually updates 10 times/second wastes battery for zero informational gain. The fix was architectural, not cosmetic: a dedicated **native 10 Hz pipeline** (`([1, 20, 6])` input, 20-sample/2.0s window, 10-sample/1.0s stride) built and trained specifically for the vehicle model, rather than forcing it through the pedestrian model's contract.

3. A ground-truth heading leak was inflating blackout-accuracy numbers. The outage-simulation evaluation code was found to be reading *live* ground-truth heading through the simulated blackout window, rather than freezing heading knowledge at the exact moment the blackout began (as a real blackout would force) — silently making the pipeline look better than it would in a genuinely blind scenario. Found and fixed as part of the same evaluation-methodology audit that produced the honest 224-evaluation benchmark in §11.

4. A real windowing-contract mismatch was caught before it could silently corrupt an evaluation. `(gru_local.onnx)` (OxIOD, `([1, 200, 6])`) and `(gru_io_vnbd.onnx)` (IO-VNBD, `([1, 20, 6])`) do not share an input shape, but the app's shared `(ModelMetadata)` object hardcoded a single global window-size constant matching only the vehicle model. This was specifically checked, directly against each model's real ONNX graph `(onnxruntime.InferenceSession(...).get_inputs())`, rather than trusted from documentation

— feeding `gru_local.onnx` a 20×6 window instead of its real 200×6 contract wouldn't have thrown an error, it would have silently run and produced meaningless output, which is a uniquely dangerous kind of bug.

5. It was discovered — and clearly documented, not glossed over — that only one of the two trained models is actually wired into the shipped app. A repo-wide grep for any reference to `gru_local` outside its own metadata file returned zero hits in Kotlin source; `ModelRunner.kt` hardcodes `gru_io_vnbd.onnx` as its only load target. This is exactly the §6.6 finding — a real, trained, validated pedestrian model exists, but describing the live app as running "two fused ONNX models" would not be accurate, and the project's own status log explicitly lists this as a claim *not* to make to a judge.

6. Plain physics beating ML on most benchmark configurations — investigated as a possible bug first, then accepted as a genuine finding. Before accepting "ML sometimes loses to plain EKF" at face value, the team specifically re-ran the plausibility gate logic *inside* the offline benchmark harness itself, to rule out "the gate is misconfigured in evaluation but fine in the app" as an explanation — it made no difference. A follow-up direction-vs-magnitude diagnostic (checking separately whether the model's predicted *direction* was reasonable versus whether its predicted *magnitude* was reasonable) showed the model wasn't systematically broken in one obvious way — it was genuinely inconsistent, doing well on some real sequences and badly on others, which is a data/generalization finding, not a code defect. This distinction — "did we rule out a bug before accepting an uncomfortable result" — is exactly the kind of thing worth being able to say plainly if asked.

7. A related bug: when the ML gate rejected a prediction, the app froze the vehicle in place instead of falling back to physics. Caught and flagged for fixing: a `REJECTED` gate outcome should mean "ignore ML for this step, keep going on physics/IMU alone" (per §6.8's whole design intent), not "stop updating position entirely" — a subtle but important distinction, since the latter would make the on-screen marker visibly freeze during exactly the moments the system should be gracefully leaning harder on its physics fallback.

8. DR Distance stuck at 0.0 m in normal (non-blackout) GPS mode. Root cause: `DeadReckoningEngine.correctWithGnss()` was updating position from real GPS fixes but never feeding those points into `trajectoryIntegrator` (the component responsible for accumulating the on-screen "distance travelled" figure) — fixed by adding the missing `addPoint()` call, scoped only to normal (non-blackout) mode via the function's existing early-return guard.

9. DR Distance stuck at 0.0 m specifically *during* blackout, even while the corrected position genuinely moved. Traced to an ordering bug: `NavigationEngine.setBlackoutMode(true)` was snapshotting the "distance at blackout start" baseline **before** calling `deadReckoningEngine.initialize(...)`, and `initialize()` itself resets the running distance total to zero — so the baseline captured was a stale, non-zero pre-

blackout total, and the on-screen figure ($\max(0, \text{currentTotal} - \text{staleBaseline})$) stayed pinned at zero until the new blackout's own total managed to exceed that stale number, which on a short blackout it never did. Fixed by moving the one-line snapshot to *after* `initialize()` runs — confirmed afterward on a real device, where "DR Distance" correctly began reading a real, moving number (5.5 m) instead of 0.0 m.

10. A ZUPT false-positive timing bug caused "Motion: STATIONARY" while genuinely walking. The stationary-confirmation debounce was implemented as a raw *sample count* (`minConsecutiveSamples = 4`), documented in an old comment as "~400ms at 10Hz" — but the function driving it was actually being called at close to 100Hz (roughly 10× the assumed rate, since it's called once per accelerometer *or* gyroscope callback, both registered at `SENSOR_DELAY_GAME`), so 4 samples confirmed "stationary" in roughly 40–80ms, not 400ms — comfortably inside the brief low-acceleration moment that occurs *within a single walking stride*, between a step's propulsion and braking phases. This let a steadily-carried phone falsely latch into `STATIONARY` and stay there while a person was genuinely walking, freezing the DR distance. Fixed by rewriting the debounce to measure real elapsed time from each sample's own monotonic timestamp (`minStationaryDurationMs = 400f`) rather than counting samples — restoring the originally-intended ~400ms debounce regardless of the sensor's actual real-world callback rate. Verified afterward against a real 49.5-second captured walk: every real transition into `STATIONARY` carried sensor values clearly, non-borderline below threshold, with no evidence of a remaining false positive in that capture.

11. A genuine sign-inversion bug found in the manual gyroscope-integration heading fallback. While investigating a user report of the on-screen heading arrow appearing to turn the wrong way, a real bug was found in `SensorFusionManager`'s fallback complementary-filter path (the one used only before the hardware rotation-vector sensor delivers its first reading, or continuously on a device lacking one): it was integrating $\text{fusedAzimuth} += \text{gyroscope}[2] * dt$, but Android defines gyroscope-Z as positive *counter-clockwise* (right-hand rule) while compass heading is defined as increasing *clockwise* — meaning this fallback path was integrating turns in the mathematically opposite rotational sense. Fixed to $\text{fusedAzimuth} -= \text{gyroscope}[2] * dt$, with the honest caveat also documented that this specific path likely wasn't the active cause on the particular test device used (which has a working hardware rotation-vector sensor, bypassing this fallback entirely) — a real, independently-verifiable bug fix, reported without overclaiming it as *the* fix for that particular symptom, alongside a rate-limited diagnostic log added specifically so the next real walking test can settle the remaining open question with actual data rather than another guess.

12. Assorted UI/UX fixes, each grounded in a specific real report rather than guessed: a free-pan camera bug (the map was forcibly recentering on every frame, fighting the user's own drag/pinch gestures) fixed via an `isFollowingUser` state flag that only re-locks on an explicit recenter-button tap or app launch; missing street-name labels on numbered highways, fixed

by adding an OSM [\(ref\)](#) (route number) fallback to the map style's label text-field coalesce chain when no name was present; and a labeling addition (rather than a behavior change) — a small always-visible legend clarifying which trail color is the corrected estimate versus the deliberately-uncorrected naive reference, added after a report that the raw naive trail's dramatic drift looked like a bug rather than the intentional demonstration it actually is.

13. Open Challenges and Honest Limitations

Stated plainly, because a judge will respect disclosed limitations far more than an unqualified claim that turns out not to hold up:

- **Phone-grade IMU noise is a real, physical ceiling.** Automotive-grade IMUs are simply better instruments than what's inside a consumer smartphone; no amount of software cleverness fully closes that gap, which is exactly why the project sets accuracy expectations using literature-grounded metrics (ATE/RTE/CEP/drift-rate) rather than promising GPS-level precision.
- **The navigation filter is an explicitly simplified 6-state MVP**, not a full 15-state error-state INS — it does not separately estimate accelerometer bias, gyroscope bias, or attitude error as part of its own state. This is a real, disclosed scope decision (get a working, understandable filter shipped and correctly reasoned about, rather than a more complete filter that's harder to verify in the time available), not an unnoticed gap.
- **Heading, in production, currently relies on the phone's own fused rotation-vector sensor** (which itself depends on magnetometer quality — degraded near metal, rebar, or vehicle chassis, exactly the environments this project targets). The offline 224-evaluation benchmark, by contrast, used the vehicle's own reference heading from the CAN bus/dual-antenna GPS to isolate pure displacement dead-reckoning performance from consumer-gyro heading drift — meaning the benchmarked numbers describe displacement-estimation quality under a *trusted* heading reference, and real unassisted deployment will see some additional heading-driven degradation over long (60-120s) blackouts that the benchmark numbers alone don't fully capture. This gap is explicitly named in the project's own frozen documentation, not discovered by an outsider.
- **Generalization across drivers and routes is real and unresolved**, not glossed over — the same model performs very differently on different real driving sequences (9.10% drift on the best moving drive at 120s versus 72.77% on the worst), and the honest, current mitigation is the kinematic plausibility gate limiting how much a bad prediction can hurt, not a claim that the underlying generalization gap has been closed.
- **Motion-domain mismatch:** the deployed model was trained specifically on data resembling dashboard-mounted vehicle motion; pedestrian, in-pocket, handbag, and running dynamics have not been benchmarked against it (a separate model exists for handheld pedestrian motion but isn't wired into the app — see §6.6), and there's no

dynamic online phone-mount self-calibration — the system currently assumes a fixed, known mounting.

- **The pedestrian carry-pose problem remains genuinely open**, per §12 item 11: raw device compass heading is only a reliable proxy for walking direction when the phone is held in a fairly flat, consistent pose, and a phone naturally swinging/tilting in a hand while walking can produce real, physically-correct heading swings that don't track actual walking direction — this affects both the on-screen marker's apparent direction and the naive trail's integration direction, and the current diagnosis-in-progress is via real logged data rather than another guessed fix.
- **Map coverage is limited to one bundled demo region** (Coimbatore, offline vector tiles) — genuinely real and reasonably capable (live marker, dual-trail blackout visualization, real offline rendering), but not yet a general-purpose coverage claim.
- **No CAN-bus / wheel-tick access in the real deployed product** (by design — this is a smartphone-only solution, matching the problem statement's own scope), which is precisely the harder, still-actively-researched version of this problem compared to the automotive Tier-1 solutions discussed in §3.

14. Jury / Judge Q&A Rehearsal

Q: Isn't this already solved by Google Maps or a car's built-in navigation? Not for this configuration. Mature automotive dead-reckoning (u-blox ADR, FURUNO GNSS/DR receivers, Sony's fusion modules) exists, but depends on dedicated automotive-grade hardware and often direct vehicle CAN-bus access (wheel ticks). This problem statement specifically asks for a bare-smartphone solution — no extra hardware — and that specific slice is still active, unsolved research (2021-2025 publications ongoing).

Q: Why not just rely on GPS or NavIC directly? Because no satellite system — Indian or foreign — can physically penetrate tunnels, underground structures, or dense high-rises; that's a line-of-sight limitation, not a software gap. It's also timely: NavIC currently has only three of the minimum four satellites needed for standalone positioning, following an onboard atomic clock failure in March 2026.

Q: How accurate is this, realistically? Reported honestly with standard inertial-odometry metrics (ATE, RTE, CEP, drift-rate as % of distance), benchmarked on real held-out data — not a single cherry-picked number. Median moving drift across held-out test sequences was 25.79% at typical blackout durations, with real variation by route and condition, and a widening on-screen confidence radius that's honest about growing uncertainty rather than implying false precision.

Q: Why a physics baseline plus an ML layer, instead of an end-to-end deep model? Because the physics layer is independently verifiable and provides a guaranteed floor of correctness, the ML layer's contribution can be gated per-prediction by a physical plausibility check rather

than trusted blindly, and — demonstrated directly by this project's own 224-evaluation benchmark — a learned model trained on real but limited data does not reliably beat a well-implemented physics baseline everywhere, which is exactly the kind of finding an all-or-nothing end-to-end model would hide rather than let you architect around.

Q: How do you get ground truth during a simulated blackout? Standard dead-reckoning research technique: record a real trace with GPS available the whole time, then withhold a real, already-recorded window of GPS labels to simulate a blackout, and measure drift against the real, withheld truth afterward. The underlying accelerometer/gyroscope data is always genuinely recorded — only the choice of which window counts as "blackened out" is simulated. (See §8.)

Q: Is any of the data synthetic? No. Every accelerometer, gyroscope, and GPS/CAN reading used is either from the published, peer-reviewed OxIOD and IO-VNBD datasets, or genuinely recorded during real testing. The only simulated element anywhere in the pipeline is the blackout window boundary for evaluation purposes.

Q: What's the actual confidence interval shown on screen — is it different for every point? It comes from the EKF's own covariance matrix, which genuinely grows during a blackout as prediction steps accumulate uncertainty without correction (§5.3) — it is a live, mathematically-derived per-moment estimate from the filter's own bookkeeping, not a fixed marketing number, though it should still be understood as the filter's *model* of its own uncertainty, not an independently verified error bound.

Q: What is ZUPT and why does it matter? A standard, well-established correction: whenever the multi-signal detector confirms the device is genuinely stationary, it feeds the filter a "velocity = zero" pseudo-measurement through the normal Kalman update (never a hard override), which meaningfully bounds the otherwise-quadratic drift that comes from double-integrating even tiny residual noise while parked or paused. (§5.4.)

Q: What's the path to real deployment beyond the hackathon? An SDK integrated into ride-hailing/logistics/delivery apps for ETA continuity through covered areas; firmware-level integration as NavIC-compatible chipsets expand; and direct further R&D handoff to ISRO, since this problem statement is itself sponsored as an ISRO research interest.

Q: Does this help with GPS spoofing or jamming, not just signal loss? Indirectly — since dead reckoning doesn't depend on receiving any GNSS signal during the blackout window at all, it's inherently unaffected by spoofing/jamming during that period, though it still needs a trustworthy fix to initialize from. Layering in a spoofing-detection trigger to fall back to dead reckoning automatically is a reasonable future direction, honestly flagged as not yet built.

15. Glossary

- **Dead reckoning:** estimating current position from a known starting point plus tracked motion since then, without an outside positioning reference. (§2)
- **GNSS:** Global Navigation Satellite System — the umbrella term covering GPS (US), NavIC (India), and other satellite positioning constellations.
- **IMU:** Inertial Measurement Unit — the combination of accelerometer + gyroscope (sometimes + magnetometer) inside a device.
- **Strapdown INS:** an inertial navigation solution where the sensors are rigidly fixed to the device itself (no mechanical stabilization), requiring all orientation/gravity correction to happen in software. (§5.2)
- **EKF (Extended Kalman Filter):** an algorithm that optimally blends a predicted estimate with new noisy measurements, weighted by how confident it is in each, producing both a best estimate and a shrinking/growing confidence value over time. (§5.3)
- **ZUPT (Zero-Velocity Update):** feeding the filter a "you're stationary" correction whenever multiple sensor signals agree the device genuinely isn't moving. (§5.4)
- **NHC (Non-Holonomic Constraint):** feeding the filter a "vehicles don't slide sideways or bounce vertically" correction during normal driving. (§5.5)
- **NED frame:** North-East-Down — a local flat-Earth coordinate system the filter's internal state is expressed in. (§5.1)
- **GRU (Gated Recurrent Unit):** a recurrent neural network design that carries a memory forward across a sequence, letting predictions depend on everything seen so far in that sequence, not just the current instant. (§6.4)
- **ONNX:** a portable file format for trained neural networks that lets a model trained in one framework (PyTorch) run efficiently in a different, lightweight runtime on a different platform (a phone), without needing the original training framework present. (§7)
- **Kinematic plausibility gate:** the component that checks whether an ML-predicted displacement implies a physically reasonable speed/speed-change before accepting it, clamping it, or rejecting it in favor of the physics fallback. (§6.8)
- **ATE / RTE / CEP / drift rate:** standard ways of quantifying navigation error — see §10 for plain-language definitions of each.
- **OxIOD:** Oxford Inertial Odometry Dataset — real pedestrian/handheld phone IMU data with Vicon motion-capture ground truth. (§6.2)
- **IO-VNBD:** Inertial and Odometry Vehicle Navigation Benchmark Dataset — real vehicle driving data with CAN-bus ground truth; the dataset the deployed model is actually trained on. (§6.2)
- **Blackout / GNSS outage:** the period during which GPS/NavIC signal is unavailable and the system relies purely on dead reckoning.

